Practical: CI/CD Pipeline for a Cloud-Native Application using GitHub, GitHub Actions and Kubernetes

23BCE093 HELI SHAH

Implemented in: https://github.com/Asc-85129/cicd-pipeline-demo • Date: 27 October 2026

1. Aim

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline for a cloud-native application using GitHub as the source repository, GitHub Actions as the automation server and Kubernetes as the deployment platform. Every push to the main branch must automatically lint and test the code and, only if all checks pass, deploy the new version to a Kubernetes cluster and verify that it is serving traffic.

Note on scope: the lab sheet also lists Docker and Docker Hub. Docker Desktop makes the lab machine too slow to work with, so this implementation deliberately runs without Docker or Docker Hub. Section 2.3 explains each substitution. Every other part of the CI/CD flow (build, test, gate, artifact, deploy, rollout, verification) is implemented and was run for real on GitHub.

Learning objectives

- Explain the difference between Continuous Integration, Continuous Delivery and Continuous Deployment.
- Write a multi-job GitHub Actions workflow with triggers, job dependencies (needs) and conditions (if).
- Use automated linting and unit tests as a quality gate that blocks a bad change from being deployed.
- Pass build outputs between jobs with workflow artifacts.
- Describe a cloud-native application declaratively with a Kubernetes Deployment, Service and ConfigMap.
- Provision a Kubernetes cluster (k3s) on a CI runner, deploy to it and verify the rollout with a smoke test.
- Understand container runtimes beyond Docker (containerd) and how to ship code without a custom image.

2. Theory

2.1 CI/CD concepts

Continuous Integration (CI) means every developer integrates code into a shared branch often, and each integration is checked automatically by a build and test run. A broken change is caught within minutes and traced to one small commit, instead of being found days later during manual integration.

Continuous Delivery keeps the main branch releasable at all times, with a person approving the final release. Continuous Deployment goes one step further: every change that passes the pipeline goes to

the target environment with no manual step. This practical implements Continuous Deployment. A green build on main is deployed straight to Kubernetes.

Concept	What it is	Role in this lab
GitHub	Git hosting service; source of truth for code and configuration	Hosts repo Asc-85129/cicd-pipeline-demo; a push to main is the pipeline trigger
GitHub Actions	CI/CD engine built into GitHub; workflows are YAML files in .github/workflows/	Runs ci-cd.yml on hosted ubuntu-latest runners
Workflow / Job / Step	Workflow = whole pipeline; job = set of steps on one runner; step = one command or action	2 jobs: build-test (CI) and deploy (CD)
needs / if	Job dependency and condition keywords	deploy needs build-test, so it is skipped when tests fail; it is also skipped for pull requests
Artifact	File produced by a job and stored by GitHub between jobs	app-release (app.py, requirements.txt, VERSION) is passed from CI to CD
Kubernetes	Container orchestrator; desired state is declared in YAML and reconciled by controllers	Runs 2 replicas of the app behind a NodePort Service
k3s	Lightweight, CNCF-certified Kubernetes distribution packaged as a single binary	Cluster started inside the CD job in about 15 s
containerd	Industry-standard container runtime (the one Docker itself uses underneath)	Runtime used by k3s; pulls and runs the python image without Docker
ConfigMap	Kubernetes object holding non-secret data, mountable as files	Carries app.py and requirements.txt into the pods at /app
Rollout / probes	Deployment rolling update; readiness and liveness probes decide pod health	kubectl rollout status waits until both pods pass /health

2.2 Pipeline stages

- **Source -** the developer pushes a commit to main on GitHub.

- **Build / Lint -** dependencies are installed on a fresh runner and flake8 checks style and syntax errors.
- **Test -** pytest runs the unit tests against the Flask app using its test client.
- **Package -** the deployable files plus a VERSION file (short commit SHA) are uploaded as an artifact.
- **Deploy -** a Kubernetes cluster is prepared, the code is published as a ConfigMap and the manifests are applied.
- **Verify -** the pipeline waits for the rollout and calls the live service; any failure turns the run red.

2.3 Running without Docker and Docker Hub

In the usual version of this lab, CI runs docker build, pushes the image to Docker Hub, and Kubernetes pulls that image. Docker is not actually required for any of these jobs, so each one was replaced:

Usual tool	Purpose	Replacement used here	Why it is equivalent
docker build	Package app + runtime into an image	Stock image python:3.11-slim + code in a ConfigMap	The pod still runs the exact committed code on a pinned Python runtime
Docker Hub	Store the versioned build output	GitHub Actions artifact app-release (+ VERSION = commit SHA)	Versioned, immutable build output handed from CI to CD
Docker Engine on the node	Run containers	containerd (built into k3s)	Kubernetes talks to containerd through the CRI; Docker support was removed from Kubernetes in v1.24 anyway
kind / Minikube on laptop	Local cluster	k3s installed on the GitHub runner	Real Kubernetes API (v1.36), nothing runs on the lab machine

Trade-off: installing Flask at pod start-up (pip install in the container command) adds a few seconds to start-up and needs internet access. That is fine for a lab. In production the image would be built once and stored in a registry. The change is small: add a build-and-push job and replace the image line in deployment.yaml.

3. Requirements

Item	Version / detail

OS	Windows 11 Home (lab machine), Git Bash
Git	2.x for Windows (see Figure 1)
Python	3.11.8 locally, 3.11.17 on the runner; Flask 3.0.3, pytest, flake8
GitHub CLI	gh, logged in as Asc-85129 (to create the repo and read runs/logs)
GitHub Actions runner	ubuntu-latest (Ubuntu 24.04.5 LTS, kernel 6.17 azure)
Kubernetes	k3s v1.36.5+k3s1, container runtime containerd 2.3.4
Browser	Microsoft Edge (headless) for the GitHub page screenshots
Not used	Docker Desktop, Docker Engine, Docker Hub

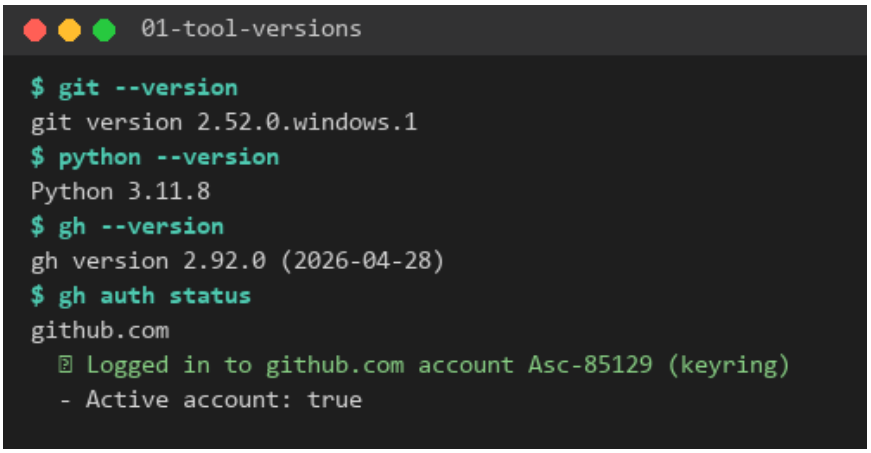


Figure 1: Local tool versions and GitHub CLI authentication

4. Architecture

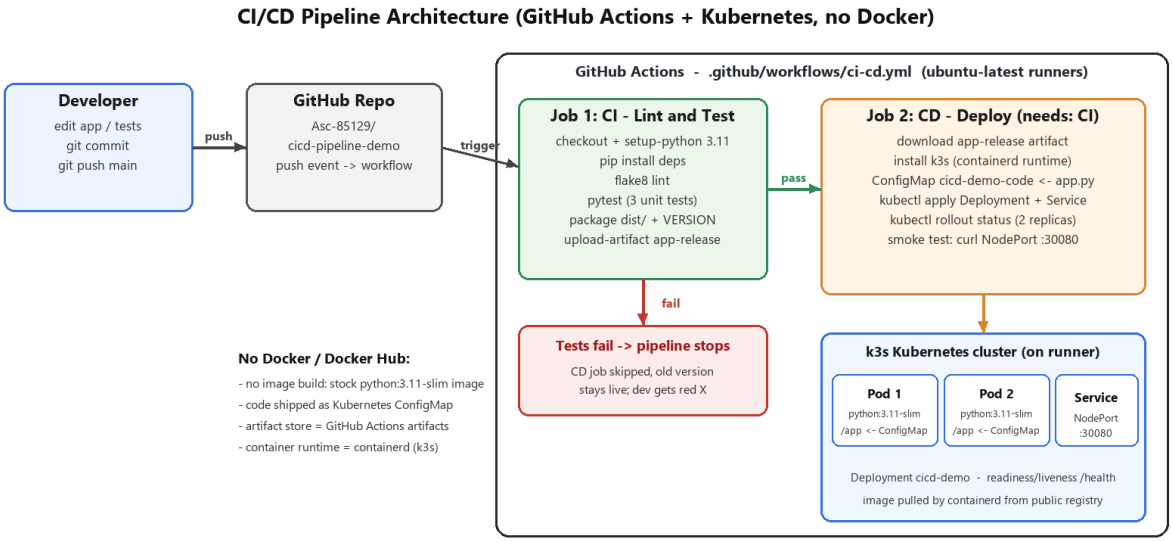


Figure 2: Pipeline architecture: CI job gates the CD job, which deploys to k3s on the runner

A push to main triggers the workflow. Job 1 (CI) runs on a fresh Ubuntu runner: it installs dependencies, lints, tests and uploads the release artifact. Job 2 (CD) starts only if Job 1 succeeded. It runs on a second, clean runner, downloads the artifact, installs k3s, stores the code in a ConfigMap, applies the Deployment and Service, waits for both replicas to become Ready and then smoke-tests the NodePort. If any CI step fails, the CD job is skipped and the version that was already deployed stays as it is.

5. Procedure

All work was done in D:\DevOps\cicd-pipeline. Final layout of the repository:

02-repo-structure

```
$ tree (git ls-files)
.github/workflows/ci-cd.yml
.gitignore
README.md
app/app.py
app/requirements.txt
k8s/deployment.yaml
k8s/service.yaml
tests/test_app.py
```

Figure 3: Files tracked in the repository

Step 1 – Write the cloud-native application (app/app.py)

A small stateless Flask service following twelve-factor rules: its configuration (port, version) comes from environment variables, it exposes a /health endpoint for Kubernetes probes, and it returns the pod hostname so that load-balancing across replicas can be seen.

```
"""Cloud-native demo service for the CI/CD practical."""
import os
import socket

from flask import Flask, jsonify

app = Flask(__name__)
VERSION = os.environ.get("APP_VERSION", "dev")


@app.route("/")
def index():
    return jsonify(
        message="Hello from CI/CD pipeline v2!",
        version=VERSION,
        pod=socket.gethostname(),
    )


@app.route("/health")
def health():
    return jsonify(status="ok")


def add(a, b):
    return a + b


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

Step 2 – Write unit tests (tests/test_app.py)

```
import os
import sys

sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "app"))

from app import add, app  # noqa: E402


def client():
    app.config["TESTING"] = True
    return app.test_client()


def test_index_returns_message():
    resp = client().get("/")
    assert resp.status_code == 200
    assert resp.get_json()["message"] == "Hello from CI/CD pipeline v2!"


def test_health():
    resp = client().get("/health")
    assert resp.status_code == 200
```

```
    assert resp.get_json() == {"status": "ok"}


def test_add():
    assert add(2, 3) == 5
```

The tests and the linter were run locally before pushing, so the code entering CI was already known to pass:

```
pip install -r app/requirements.txt pytest flake8
python -m flake8 app tests --max-line-length=100
python -m pytest -v tests
```

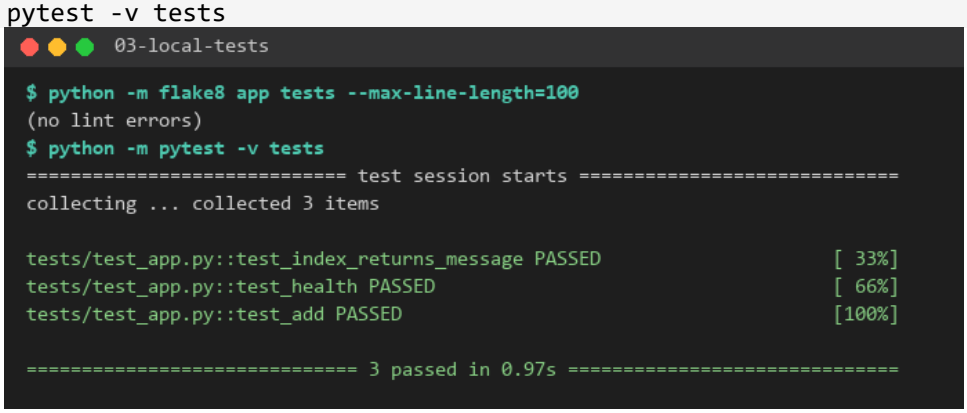


Figure 4: Local run: flake8 clean, 3/3 tests passed

Step 3 – Write the Kubernetes manifests (k8s/)

deployment.yaml: 2 replicas of the stock python:3.11-slim image. The ConfigMap cicd-demo-code is mounted at /app. The container installs the requirements and starts the app. APP_VERSION_PLACEHOLDER is replaced with the commit SHA at deploy time. The readiness probe keeps a pod out of the Service until /health answers, and the liveness probe restarts it if it hangs.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cicd-demo
  labels:
    app: cicd-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cicd-demo
  template:
    metadata:
      labels:
        app: cicd-demo
    spec:
      containers:
        - name: web
          # Stock public image - no custom image build / registry needed.
          image: python:3.11-slim
          command: ["sh", "-c"]
```

```
          args:
            - pip install --no-cache-dir -r /app/requirements.txt && python
/app/app.py
          env:
            - name: APP_VERSION
              value: "APP_VERSION_PLACEHOLDER"
          ports:
            - containerPort: 5000
          volumeMounts:
            - name: app-code
              mountPath: /app
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 30
            periodSeconds: 10
      volumes:
        - name: app-code
          configMap:
            name: cicd-demo-code
```

service.yaml: a NodePort Service that load-balances port 30080 on the node to port 5000 on every pod with label app=cicd-demo.

```
apiVersion: v1
kind: Service
metadata:
  name: cicd-demo
spec:
  type: NodePort
  selector:
    app: cicd-demo
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

Step 4 – Write the GitHub Actions workflow (.github/workflows/ci-cd.yml)

Key design points: triggers on push and pull_request to main, plus a manual workflow_dispatch. needs: build-test makes deployment depend on a green CI job. if: github.event_name != 'pull_request' stops pull requests from deploying. The short SHA is written to VERSION, so each deployment can be traced to its commit. The smoke test checks that the live service returns that exact version. A failure-only step prints pod descriptions and logs for debugging.

```
name: CI-CD Pipeline

on:
```

```
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

jobs:
  # ---------------- Continuous Integration ----------------
  build-test:
    name: CI - Lint and Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r app/requirements.txt
          pip install pytest flake8

      - name: Lint with flake8
        run: flake8 app tests --max-line-length=100

      - name: Run unit tests
        run: pytest -v tests

      - name: Package release artifact
        run: |
          mkdir -p dist
          cp app/app.py app/requirements.txt dist/
          echo "${GITHUB_SHA::7}" > dist/VERSION

      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: app-release
          path: dist/

  # ---------------- Continuous Deployment ----------------
  deploy:
    name: CD - Deploy to Kubernetes (k3s)
    needs: build-test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Download release artifact
        uses: actions/download-artifact@v4
```

```
        with:
          name: app-release
          path: dist/

      - name: Start Kubernetes cluster (k3s, containerd - no Docker)
        run: |
          curl -sfL https://get.k3s.io | sh -s - --write-kubeconfig-mode 644
          mkdir -p ~/.kube && cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
          kubectl wait --for=condition=Ready node --all --timeout=120s
          kubectl get nodes -o wide

      - name: Publish app code as ConfigMap
        run: |
          kubectl create configmap cicd-demo-code \
            --from-file=dist/app.py --from-file=dist/requirements.txt \
            --dry-run=client -o yaml | kubectl apply -f -

      - name: Deploy manifests
        run: |
          VERSION=$(cat dist/VERSION)
          sed "s/APP_VERSION_PLACEHOLDER/$VERSION/" k8s/deployment.yaml | kubectl
apply -f -
          kubectl apply -f k8s/service.yaml

      - name: Wait for rollout
        run: |
          kubectl rollout status deployment/cicd-demo --timeout=180s
          kubectl get deploy,pods,svc -o wide

      - name: Smoke test
        run: |
          for i in 1 2 3; do curl -s http://localhost:30080/; echo; done
          curl -sf http://localhost:30080/health
          curl -s http://localhost:30080/ | grep -q "$(cat dist/VERSION)"
          echo "Smoke test passed"

      - name: Debug on failure
        if: failure()
        run: |
          kubectl describe pods -l app=cicd-demo || true
          kubectl logs -l app=cicd-demo --tail=50 || true
```

Step 5 – Create the GitHub repository and push

```
git init -b main
git add .
git commit -m "CI/CD pipeline: Flask app, tests, k8s manifests, GitHub Actions
workflow"
gh repo create Asc-85129/cicd-pipeline-demo --public --source . --push
```

The push immediately triggered workflow run #1. No secrets are needed, because nothing is pushed to an external registry.

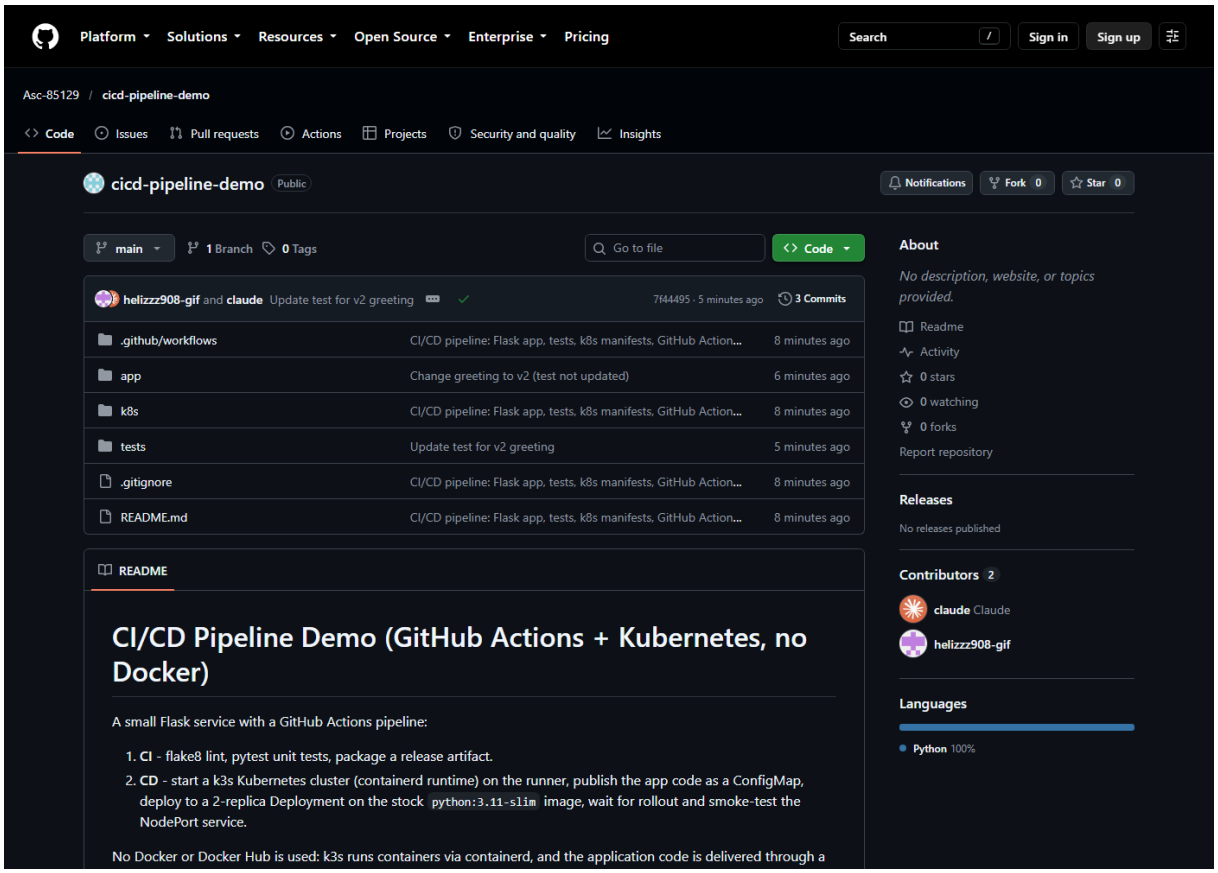


Figure 5: Repository on GitHub after the first push

6. Execution and Output

6.1 Run #1 – first commit, full pipeline green

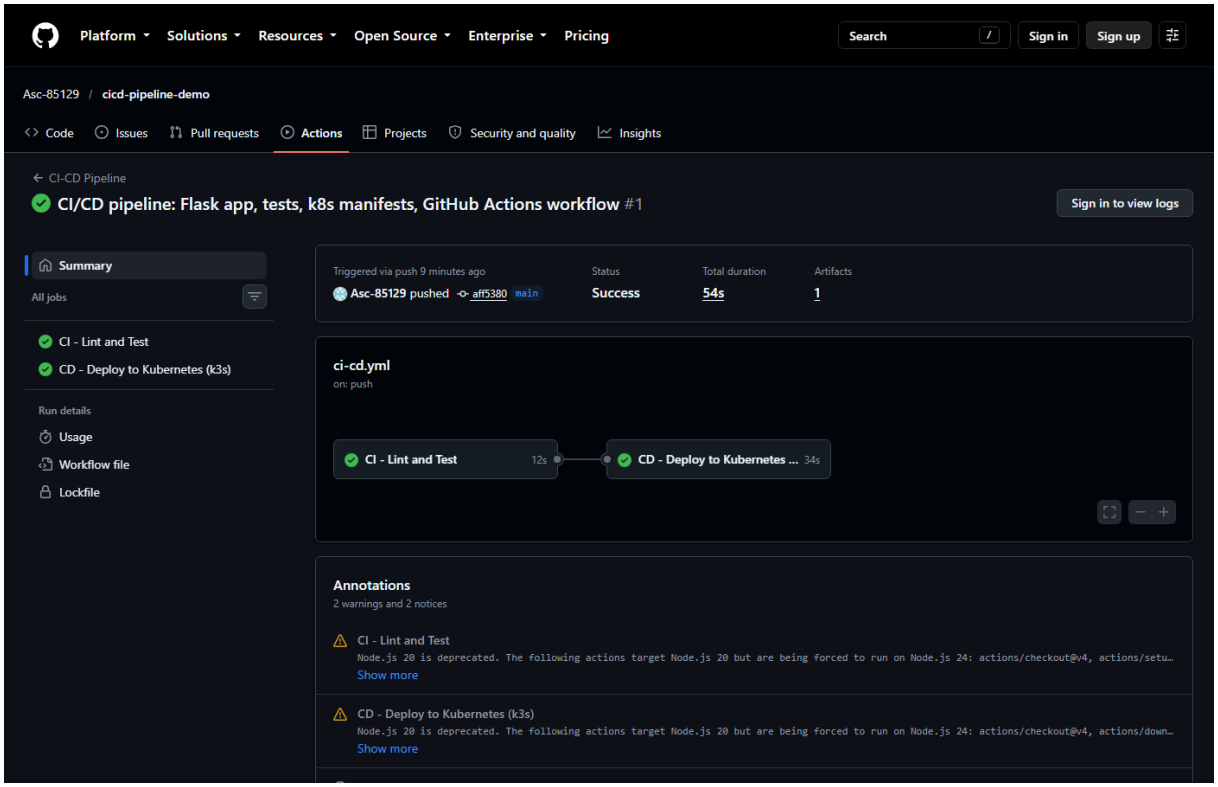


Figure 6: Run #1 summary: CI (12 s) followed by CD (34 s), status Success, 1 artifact

CI job log: flake8 reports nothing, and pytest passes all three tests on the runner:

05-ci-job-log

```
$ flake8 app tests --max-line-length=100
$ pytest -v tests
============================= test session starts ==============================
platform linux -- Python 3.11.17, pytest-9.1.1, pluggy-1.7.0 -- /opt/hostedtoolcache/Python/3.11.17/x64/bin/python
cachedir: .pytest_cache
rootdir: /home/runner/work/cicd-pipeline-demo/cicd-pipeline-demo
collecting ... collected 3 items
tests/test_app.py::test_index_returns_message PASSED                     [ 33%]
tests/test_app.py::test_health PASSED                                    [ 66%]
tests/test_app.py::test_add PASSED                                       [100%]
============================== 3 passed in 0.10s ===============================
```

Figure 7: CI job: lint and unit tests on the GitHub runner

CD job: k3s installs and the node becomes Ready in a few seconds. Note the CONTAINER-RUNTIME column, containerd://2.3.4-k3s1.36. No Docker is involved.

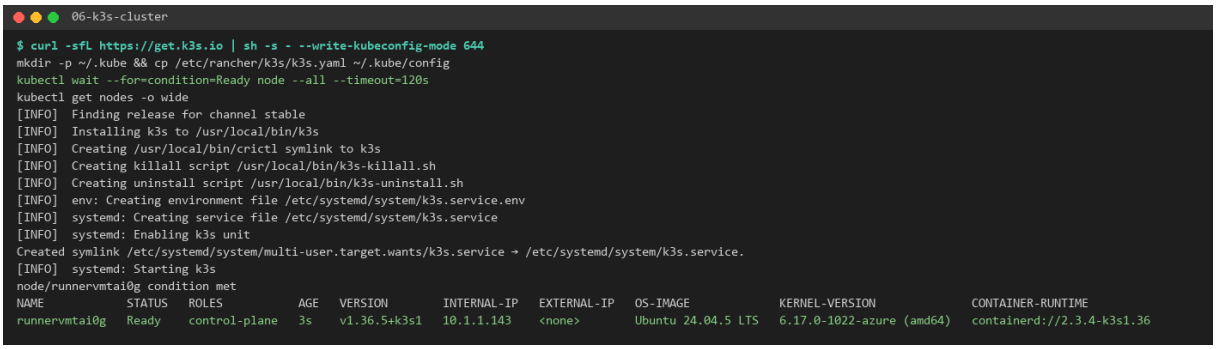


Figure 8: k3s cluster created on the runner, node Ready, runtime containerd

The ConfigMap, Deployment and Service are created. kubectl rollout status waits until both replicas are available, and then the Deployment shows 2/2 Ready with 2 pods Running:

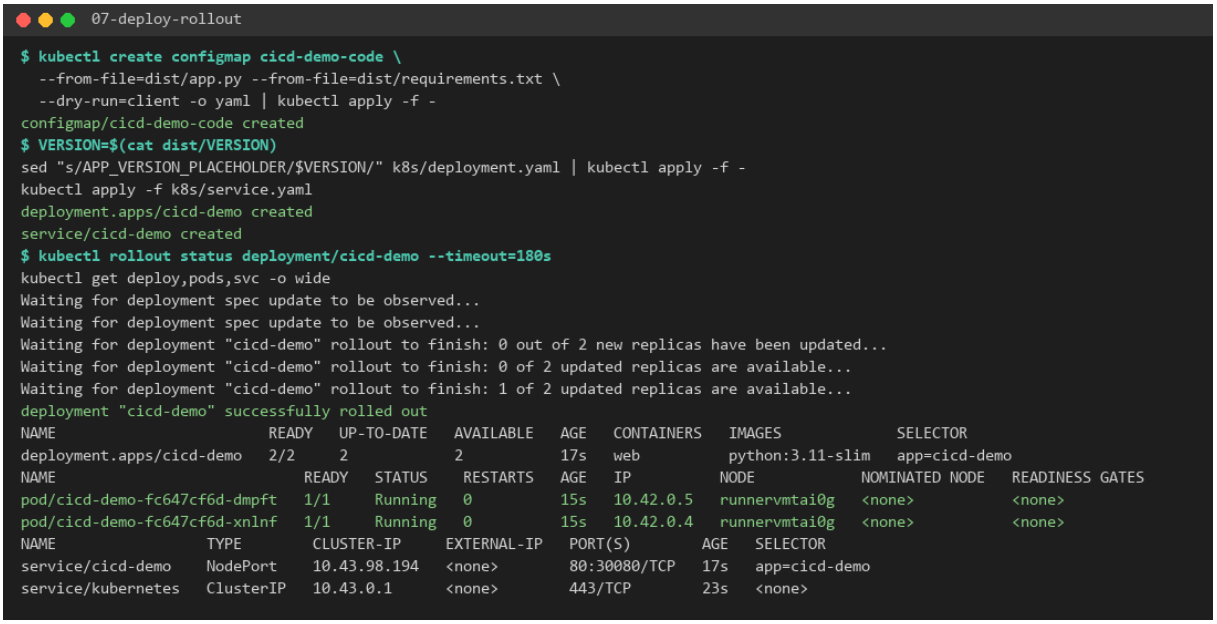


Figure 9: ConfigMap + manifests applied, rollout completes, 2/2 pods Running, NodePort 80:30080

Smoke test: three requests to the NodePort are answered by both pods (dmpft and xnlnf), which shows the Service is load-balancing. Each reply carries version aff5380, the commit that was pushed, and /health returns ok:



Figure 10: Smoke test against the live service

6.2 Run #2 – quality gate blocks a broken change

To show that the pipeline really works as a gate, the greeting in app.py was changed to "Hello from CI/CD pipeline v2!" without updating the test, and the change was pushed:

```
sed -i 's/Hello from the CI\/CD pipeline!/Hello from CI\/CD pipeline v2!/' app/app.py
git commit -am "Change greeting to v2 (test not updated)"
git push
```

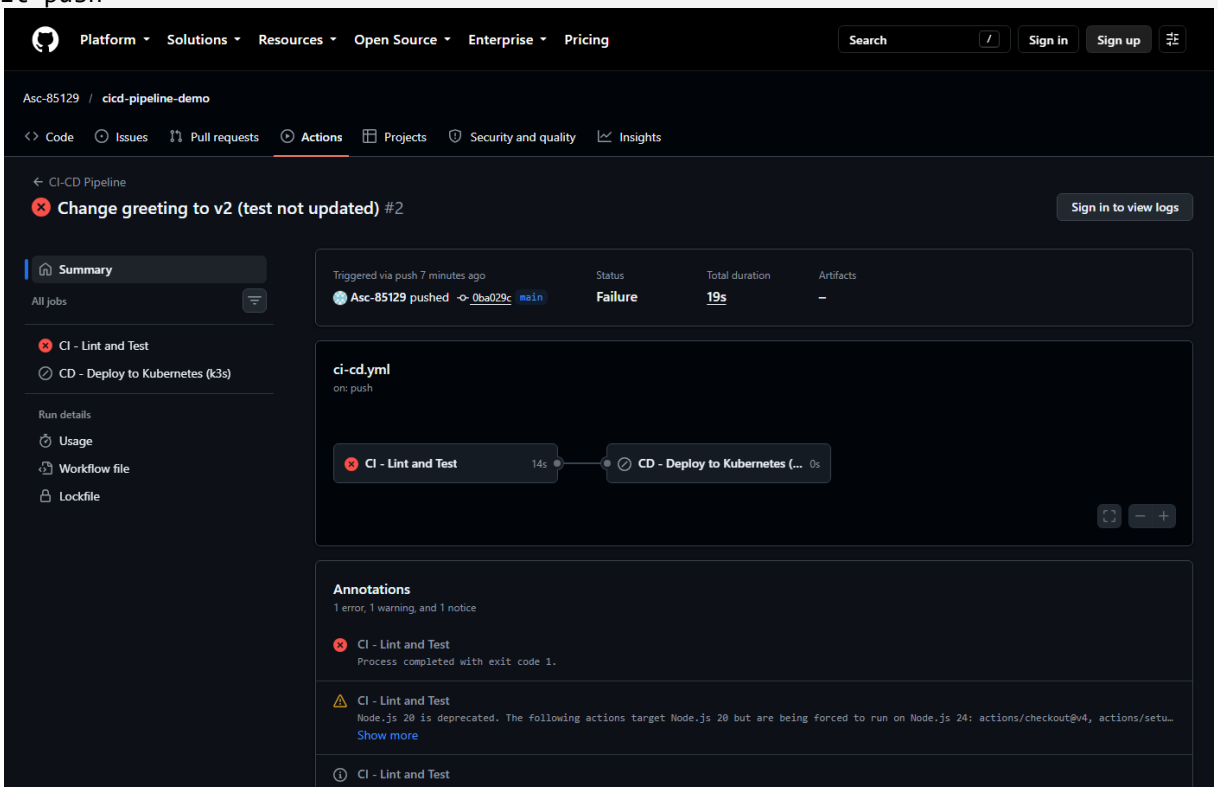


Figure 11: Run #2: CI failed, CD job skipped

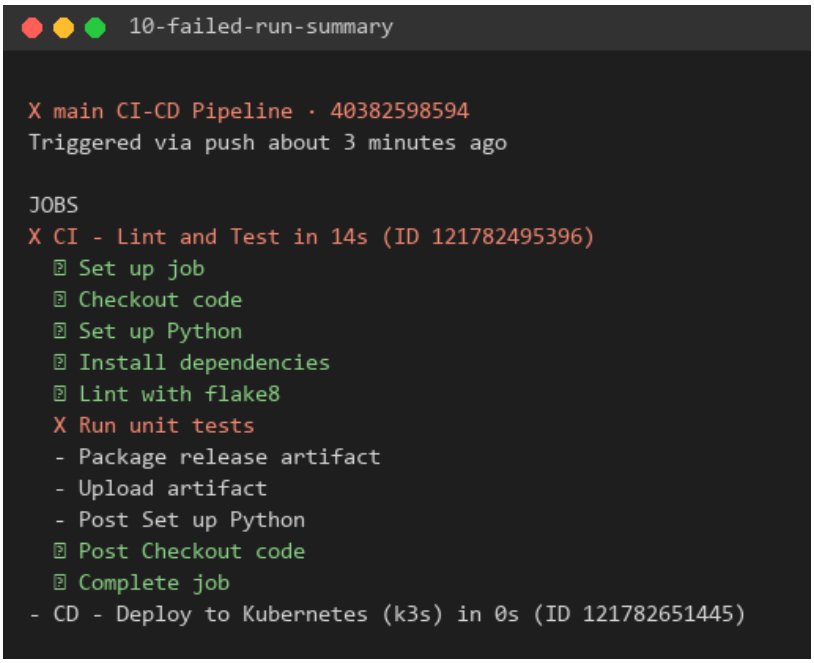


Figure 12: gh run view: the Run unit tests step failed, later steps and the deploy job skipped

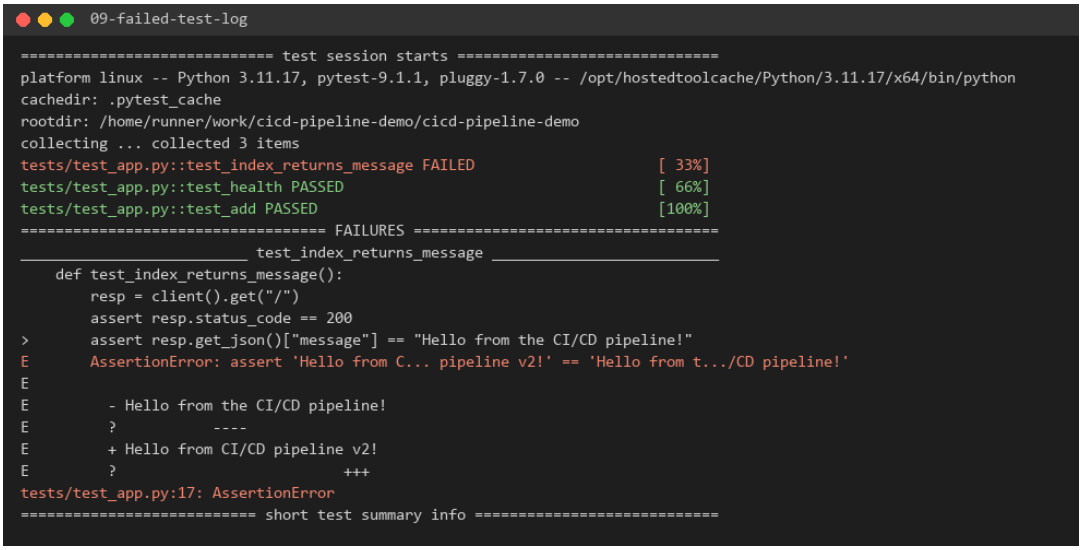


Figure 13: pytest output on the runner: assertion mismatch on the message field

Because deploy has needs: build-test, the deploy job was skipped and never ran. In a long-lived environment, the previous good version would still be serving users.

6.3 Run #3 – fix and redeploy v2

```
sed -i 's/Hello from the CI\/CD pipeline!/Hello from CI\/CD pipeline v2!/'
tests/test_app.py
python -m pytest -q tests        # 3 passed
git commit -am "Update test for v2 greeting"
git push
```

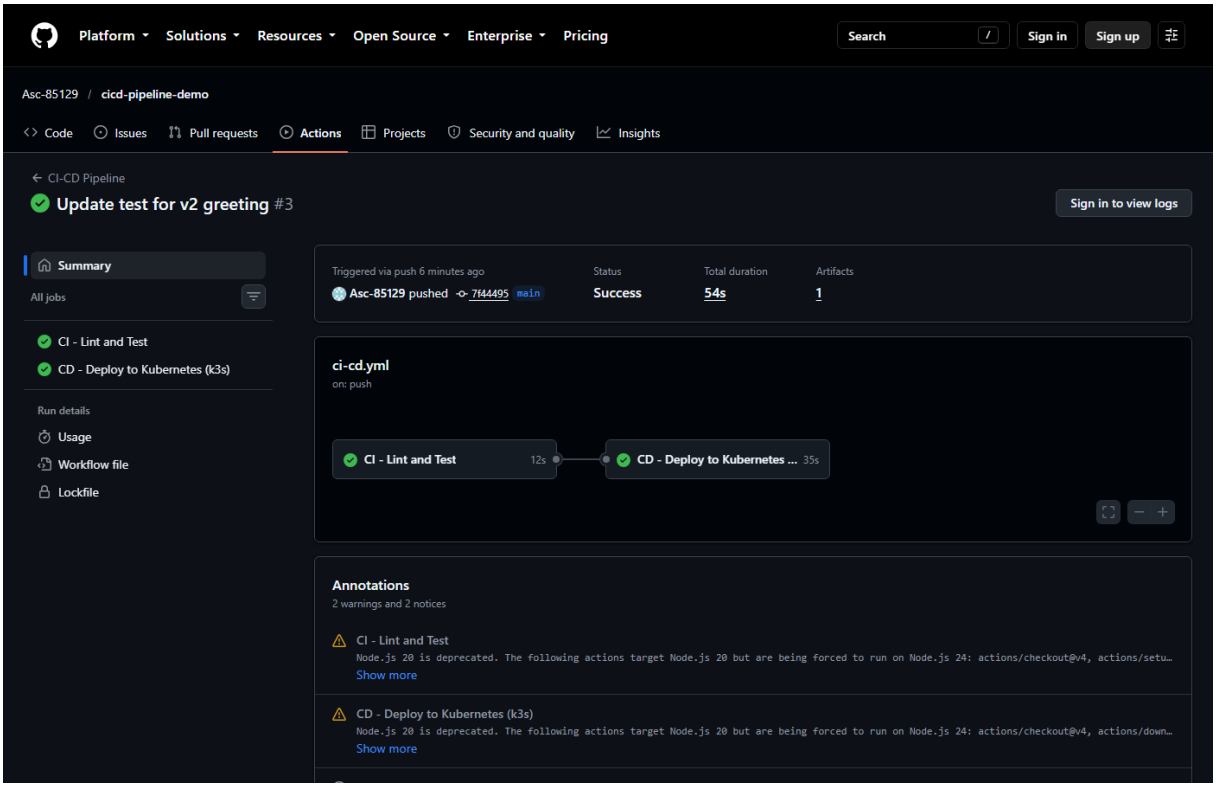


Figure 14: Run #3: both jobs green

```
11-v2-deploy
$ gh run view 40382747880   # v2 deploy

✓ main CI-CD Pipeline · 40382747880
Triggered via push about 2 minutes ago

JOBS
✓ CI - Lint and Test in 12s (ID 121783017546)
✓ CD - Deploy to Kubernetes (k3s) in 35s (ID 121783159516)

--- Smoke test output ---
deployment "cicd-demo" successfully rolled out
echo "Smoke test passed"
{"message":"Hello from CI/CD pipeline v2!","pod":"cicd-demo-79c6cf49d6-2fnlt","version":"7f44495"}
{"message":"Hello from CI/CD pipeline v2!","pod":"cicd-demo-79c6cf49d6-2fnlt","version":"7f44495"}
{"message":"Hello from CI/CD pipeline v2!","pod":"cicd-demo-79c6cf49d6-2fnlt","version":"7f44495"}
{"status":"ok"}
Smoke test passed
```

Figure 15: Run #3 smoke test: live service returns the v2 message with version 7f44495

The new version was deployed with no manual step: the change went from git push to a verified running deployment in under a minute.

6.4 Run history

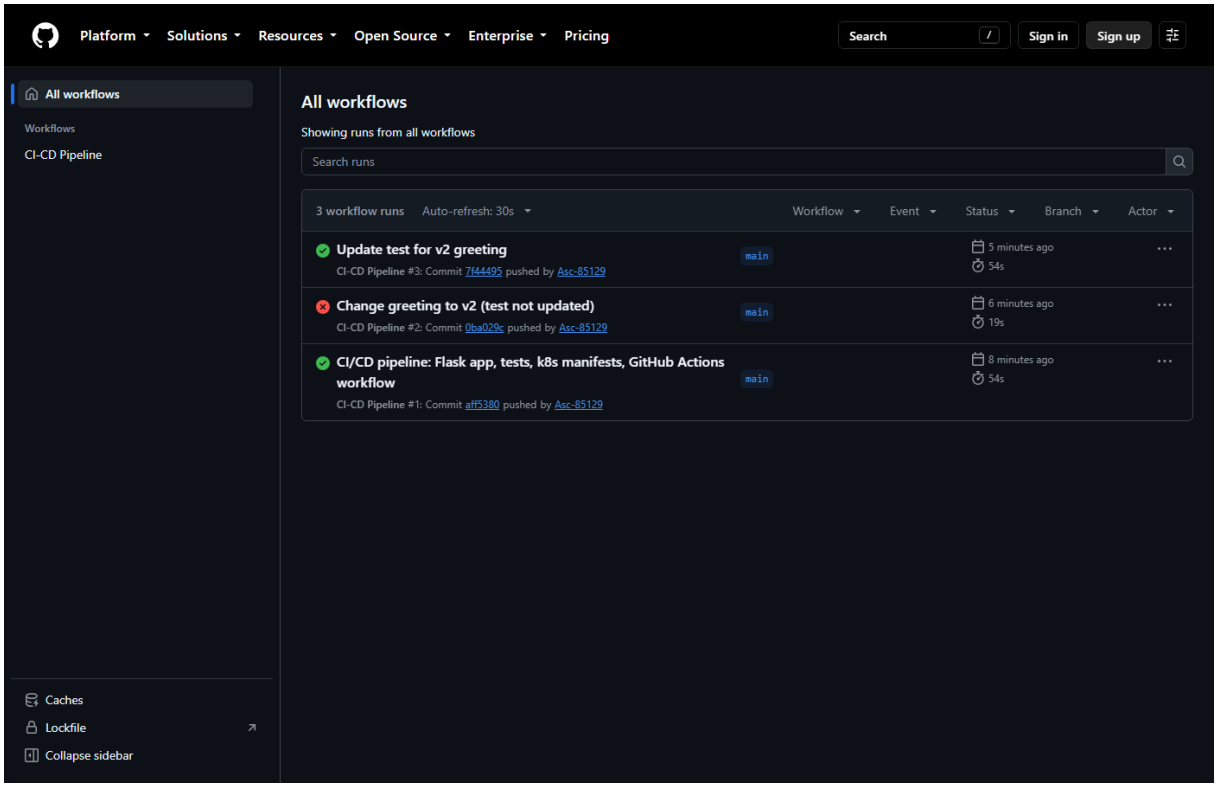


Figure 16: GitHub Actions run list for the workflow

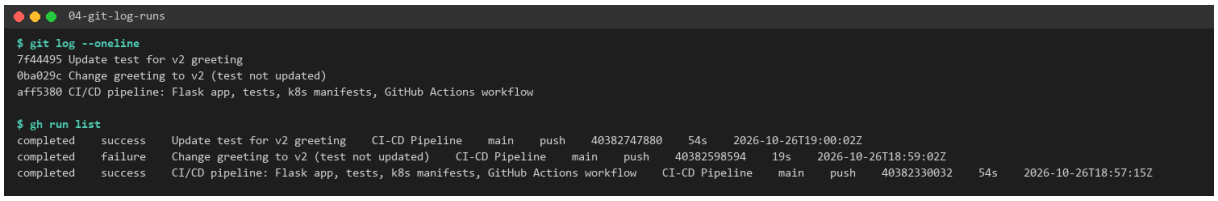


Figure 17: Commits and their matching workflow runs

7. Pipeline Run Summary

Run	Commit	Change	CI	CD	Outcome
#1	aff5380	Initial app, tests, manifests, workflow	Pass (12 s)	Pass (34 s)	v1 deployed, 2/2 pods, smoke test OK
#2	(v2 greeting)	Greeting changed, test not updated	Fail (pytest)	Skipped	Bad change blocked before deployment

#3	7f44495	Test updated for v2	Pass (12 s)	Pass (35 s)	v2 deployed and verified

8. Troubleshooting

Problem	Cause	Fix
Docker Desktop makes the laptop lag	Docker Desktop runs a WSL 2 VM with large memory/CPU use	Removed Docker entirely: k3s with containerd on the GitHub runner, code delivered via ConfigMap
Docker Hub needs an account, token and repository secrets	An external registry needs credentials	Replaced with GitHub Actions artifacts. The pipeline needs no secrets at all.
Pods would fail if deployed before the code exists	The pods mount ConfigMap cicd-demo-code	The ConfigMap is created (idempotently, with --dry-run=client | apply) before the Deployment is applied
Without probes, Service traffic reaches a pod still running pip install	Kubernetes treats a container as ready as soon as it starts	Added a readinessProbe on /health. rollout status waits for both pods to be Ready.
Log captures contained ANSI colour codes and echoed commands	GitHub log format	Stripped with sed before rendering the screenshots
Annotation: Node.js 20 actions deprecated	actions/*@v4 target Node 20; GitHub forces Node 24	Warning only, the runs pass. It can be cleared later by moving to newer major versions of the actions.

9. Result

A working CI/CD pipeline was implemented with GitHub, GitHub Actions and Kubernetes. Every push to main is linted and unit-tested automatically. A green build is packaged as a versioned artifact and deployed to a real Kubernetes (k3s) cluster as a 2-replica Deployment behind a Service, and the deployment is verified by an automated smoke test that checks the commit version being served. Run #2 showed that a failing test stops deployment, and run #3 showed a fixed change reaching production automatically. Docker and Docker Hub were not used: containerd (through k3s) ran the containers, and the code was delivered through a ConfigMap and GitHub artifacts.

10. References

- GitHub Actions documentation – https://docs.github.com/actions
- Workflow syntax (needs, if, artifacts) – https://docs.github.com/actions/writing-workflows/workflow-syntax-for-github-actions
- Kubernetes documentation: Deployments, Services, ConfigMaps, probes – https://kubernetes.io/docs/
- k3s documentation – https://docs.k3s.io
- Kubernetes dockershim removal FAQ – https://kubernetes.io/blog/2022/02/17/dockershim-faq/
- Flask documentation – https://flask.palletsprojects.com
- Project repository – https://github.com/Asc-85129/cicd-pipeline-demo